

Manual Operativo de Datos

Sistema "Atención consultas compras y alquileres Inmobiliaria IA AUTOMATION" — Triage automático de consultas inmobiliarias con n8n + IA + Notion + Gmail + Slack + Telegram

1. Esquema de bases de datos de Notion

El sistema usa dos bases de datos de Notion vinculadas entre sí mediante una propiedad de tipo Relation. Una funciona como "memoria operativa" del flujo (cada consulta de cliente y su estado) y la otra como catálogo/fuente de consulta (RAG) que el agente de IA usa como herramienta.

1.1 Tabla "REQUERIMIENTOS ENTREGA 8"

Registra cada consulta de cliente recibida por Gmail, el resultado de la clasificación de la IA, el estado de aprobación humana y los identificadores necesarios para responder por el canal correcto.

Campo	Tipo Notion	Valores / Ejemplo	Origen (quién lo escribe)
CLIENTE	Title	Correo del cliente (ej: <code>cliente@gmail.com</code>)	Nodo Notion · Create page
INTENCION	Select / Texto	<code>comprar, alquilar, consulta_general, no_clara, error_api_ia</code>	AI Agent → Code (JavaScript)
CODIGO CONSULTADO	Texto	<code>VTA-003, ALQ-001</code> (normalizado a mayúsculas)	Code (JavaScript)
PRIORIDAD	Select	<code>ALTA, MEDIA, BAJA, A_REVISAR</code>	Code (JavaScript) — regla de negocio hardcodeada
DETALLE PROPIEDAD	Texto	<code>Direccion: ... Caracteristicas: ... Precio: \$...</code>	Notion Create page (armado con expresión)
PROPIEDADES RELACIONADA	Relation → Propiedades	UUID de página de la tabla Propiedades	Notion Get many (búsqueda por CODIGO) → Create page
ESTADO	Select	<code>PROCESADO POR IA, APROBADO POR HUMANO, RECHAZADO POR HUMANO, ERROR-REVISAR MANUALMENTE</code>	IA (estado inicial) / Humano (edición manual)
NOTIFICADO	Checkbox	<code>true / false</code>	Circuito 2 (evita notificaciones duplicadas — idempotencia)
FECHA	Fecha	<code>{{ \$now }}</code>	Notion Create page
GMAIL MESSAGE ID	Texto	id del mensaje de Gmail	Gmail Trigger — usado para "Remove label UNREAD"
GMAIL THREAD ID	Texto	threadId del hilo de Gmail	Gmail Trigger — usado para "Reply to message"
Error de procesamiento	Checkbox	<code>true / false</code>	Code (JavaScript) / rama de error del AI Agent

Por qué GMAIL MESSAGE ID y GMAIL THREAD ID son dos columnas distintas: la operación "Reply to a message" de Gmail necesita el threadId, mientras que "Remove label" (usado para marcar como leído) necesita el id del mensaje individual. Usar el valor equivocado en cada operación produce el error Invalid id value. Tuve que usar Remove label porque tenía problemas con la opción Marcar como leído.

1.2 Tabla "Propiedades" (catálogo / RAG)

Catálogo de inmuebles disponibles. El agente de IA la consulta como herramienta (tool) para validar códigos de propiedad y para listar opciones ante consultas generales.

Campo	Tipo Notion	Valores / Ejemplo
CODIGO	Title	VTA-003, ALQ-001 (mayúsculas, único)
OPERACION	Select	COMPRAR, ALQUILER (valores exactos en mayúsculas – deben coincidir con el prompt del agente)
TIPO DE INMUEBLE	Select	PROPIEDAD, COCHERA
DIRECCION	Text	Balcarce 80, Rosario
CARACTERISTICAS	Text	3 dormitorios, 85M ²
PRECIO	Numeros decimales/Dolar Estadounidense	150000

Relación entre tablas: la propiedad "Propiedad Relacionada" de "REQUERIMIENTOS ENTREGA 8" no guarda el texto del código (ej. "VTA-003"), sino el UUID real de la página de "Propiedades". Por eso, antes de crear el requerimiento, un nodo Notion "Get many" busca la página cuyo CODIGO coincide y toma su id para completar la relación. Esto evita datos aislados (tablas sin vínculo real)

2. Esquemas JSON de transferencia de las integraciones

Cada nodo del flujo recibe y entrega datos en un formato JSON específico. A continuación, el contrato de datos de cada punto de integración relevante.

2.1 Gmail Trigger → AI Agent

El disparador de Gmail entrega el correo entrante; el campo "text" (cuerpo plano) se pasa como mensaje de usuario al agente de IA.

```
{
  "id": "18d2f...a91",           // Message ID (para Remove label)
  "threadId": "18d2f...a91",    // Thread ID (para Reply)
  "headers": {
    "from": "cliente@gmail.com"  // usado como Title y remitente
  },
  "text": "Hola, quiero info sobre la propiedad VTA-003",
  "subject": "Consulta propiedad"
}
```

2.2 Salida del AI Agent (contrato de la IA)

El agente está forzado (Response Format: json_object en el OpenAI Chat Model) a devolver siempre este esquema, evitando texto libre que rompa el parseo posterior.

```
{
  "consultas": [
    {
      "intencion": "comprar",          // comprar | alquilar | consulta_general |
no_clara
      "codigo_propiedad": "VTA-003",  // "" si no se menciona código
      "codigo_valido": true,          // booleano crudo de la IA (NO se confía
ciegamente)
      "prioridad": "ALTA",            // sugerida por la IA, recalculada luego en Code
      "datos_propiedad": {
        "direccion": "CRESPO 1000, ROSARIO",
        "caracteristicas": "3 dormitorios, 90M²",
        "precio": "800000"
      },
      "texto_respuesta_cliente": "Hola, gracias por tu consulta..."
    }
  ]
}
```

2.3 Salida del nodo Code (JavaScript) — post-procesamiento

Sanitiza el JSON (escapa saltos de línea dentro de strings), recalcula prioridad con reglas de negocio fijas (no confía en el valor de la IA) y normaliza el código a mayúsculas.

```
{
  "intencion": "comprar",
  "codigo_propiedad": "VTA-003",
  "codigo_valido": true,          // recalculado: true solo si hay código Y la IA lo
marcó válido
  "prioridad": "ALTA",           // regla: comprar+válido=ALTA,
alquilar+válido=MEDIA,
                                  //          general o inválido=BAJA,
resto=A_REVISAR
  "datos_propiedad": { "direccion": "...", "caracteristicas": "...", "precio": "..."},
  "texto_respuesta_cliente": "...",
  "error_procesamiento": false
}
```

2.4 Rama de error (AI Agent → On Error → Code → Notion)

Si la llamada a la API de OpenAI falla, el output de error del AI Agent se transforma en un registro mínimo que igual se guarda en Notion para revisión manual (requisito de resiliencia).

```
{
  "intencion": "error_api_ia",
  "prioridad": "A_REVISAR",
  "error_procesamiento": true,
  "detalle_error": "Fallo la llamada a la API de OpenAI"
}
```

2.5 Notion · Create/Update page (payload de escritura)

Estructura simplificada de lo que n8n envía a la API de Notion al crear un requerimiento (rama "con propiedad").

```
{
  "parent": { "database_id": "<id de REQUERIMIENTOS ENTREGA 8>" },
  "properties": {
    "Title": { "title": [{ "text": { "content": "cliente@gmail.com" } } ] },
    "INTENCION": { "select": { "name": "comprar" } },
    "PRIORIDAD": { "select": { "name": "ALTA" } },
    "CODIGO_VALIDO": { "checkbox": true },
    "ESTADO": { "select": { "name": "Procesado por IA" } },
    "NOTIFICADO": { "checkbox": false },
    "FECHA": { "date": { "start": "2026-09-26T13:00:00Z" } },
    "Propiedad Relacionada": { "relation": [{ "id": "a1b2c3d4-...-uuid" } ] }
  }
}
```

2.6 Telegram · Send message (Circuito 2 — Aprobados)

Notifica al agente inmobiliario humano cuando un requerimiento fue aprobado, con el detalle de la propiedad.

```
{
  "chat_id": "-100123456789",
  "text": "Posibilidad de comprador, contactar a: cliente@gmail.com
        \\n\\nCódigo: VTA-003
        \\nDireccion: CRESPO 1000, ROSARIO | Caracteristicas: 3 dormitorios, 90M²|
Precio: $80000"
}
```

2.7 Slack · Send message (Circuito 1 — Notificación de revisión)

```
{
  "channel": "#inmobiliaria-compras-alquileres",
  "text": "Equipo de ventas revisar en BD REQUERIMIENTOS ENTREGA 8 transacción: VTA-003"
}
```

3. Flujo de datos extremo a extremo

Gmail Trigger (JSON crudo del correo) → AI Agent (consulta a herramientas Notion + genera JSON de consultas) → Code JavaScript (sanitiza y recalcula prioridad) → IF (¿hay propiedad?) → Notion Get many (resuelve UUID) → Notion Create page (persiste el requerimiento) → Switch por PRIORIDAD → Slack / Gmail Reply (salida inmediata o espera de aprobación) → [el humano edita ESTADO en Notion] → Schedule Trigger (Circuito 2) → Notion Get many (filtra Aprobado/Rechazado + NOTIFICADO=false) → Gmail Reply + Remove label + Telegram → Notion Update (NOTIFICADO=true)